GUIDE FOR POSTMAN IMPLEMENTATION

USER -> PRODUCT CREATION -> ORDER PLACED -> LOGISTICS

Auth (8081), **Catalog (8082)**, and **Order (8083)** services **1. Place the Order**

Status: CONFIRMED

- **Method:** POST
- **URL:** http://localhost:8083/api/orders/place
- **Body (JSON):**

JSON

```
{
  "userName": "Ahamed M",
  "productId": 101,
  "quantity": 1
}
```

- **Result:** Order is created in the database. orderId is generated (e.g., 1).

---

2. Move to Packing

Status: PACKED

- **Method:** PATCH
- **URL:** http://localhost:8083/api/orders/1/status
- **Body (JSON):**

JSON

```
{
  "newStatus": "PACKED"
}
```

- **Result:** Internal warehouse preparation. No communication with port 8086 yet.

---

3. The Integration Trigger (The Dispatch)

Status: SHIPPED

- **Method:** PATCH
- **URL:** http://localhost:8083/api/orders/1/status
- **Body (JSON):**

JSON

```
{
  "newStatus": "SHIPPED"
}
```

- **What happens on Port 8083:** It saves the status and immediately sends a POST request to http://localhost:8086/api/shipments/createShipment/1.
- **What happens on Port 8086:** The Logistics Service receives the call, picks a carrier, generates a tracking ID, and saves a Shipment record.

---

4. Tracking the Order (The Enrichment)

Now, you act as the customer checking the app.

- **Method:** GET
- **URL:** http://localhost:8083/api/orders/1
- **Expected Response (Port 8083 calls 8086 behind the scenes):**

JSON

```
{
  "orderId": 1,
  "orderStatus": "SHIPPED",
  "totalOrderAmount": 750.0,
  "trackingUrl": "https://mock-delhivery/track",
  "carrier": "MOCK_DELHIVERY",
  "createdAt": "2026-04-08T11:45:00"
}
```

---

5. Direct Logistics Audit (Optional)

If you want to see the raw shipping data sitting on the Logistics microservice.

- **Method:** GET
- **URL:** http://localhost:8086/api/shipments/order/1
- **Result:** Returns the full ShipmentResponse containing the shipmentId (UUID) and the ShipmentStatus (e.g., CREATED).

---

6. Delivery Confirmation

Status: DELIVERED

- **Method:** PATCH
- **URL:** http://localhost:8083/api/orders/1/status
- **Body (JSON):**

JSON

```
{
  "newStatus": "DELIVERED"
```

Phase 1: Auth Service (Port 8081)

First, we create the user account that will be used for ordering.

1. Register a New User

- **Method:** POST
- **URL:** http://localhost:8081/api/auth/register
- **Body (JSON):**

JSON

```
{
  "username": "sai2004",
  "email": "sai@example.com",
  "password": "password123",
  "role": "CUSTOMER"
}
```

2. Login (To get JWT Token)

- **Method:** POST
- **URL:** http://localhost:8081/api/auth/login
- **Body (JSON):**

JSON

```
{
  "username": "sai2004",
  "password": "password123"
}
```

- **Response:** You will get a { "token": "eyJhbG..." }. Copy this if your Order service requires it.

3. Verify User Details (The Feign Endpoint)

- **Method:** GET
- **URL:** http://localhost:8081/api/users/sai2004
- **Verification:** Ensure this returns a **200 OK** with user details. If it shows **403**, check your SecurityConfig.

Phase 2: Catalog Service (Port 8082)

Now, build the "Master Options" and the "Customized Product."

4. Create Master Option (Material)

- **Method:** POST
- **URL:** http://localhost:8082/api/options
- **Body (JSON):**

JSON

```
{
  "type": "Material",
  "value": "Italian Leather",
  "priceAdjustment": 2500.0
}
```

5. Create Master Option (Color)

- **Method:** POST
- **URL:** http://localhost:8082/api/options
- **Body (JSON):**

JSON

```
{
  "type": "Color",
  "value": "Crimson Red",
  "priceAdjustment": 400.0
}
```

6. Build the Product (Dynamic Builder)

- **Method:** POST
- **URL:** http://localhost:8082/api/products/create
- **Body (JSON):**

JSON

```
{
  "name": "Designer Leather Jacket",
  "basePrice": 2000.0,
  "previewImage": "jacket_v1.png",
  "selectedOptionIds": [1, 2]
}
```

- **Result:** You should see totalPrice: 4900.0 (2000 + 2500 + 400). Note the productId (usually **1**).

Phase 3: Order Service (Port 8083)

Place the actual order using the user from 8081 and the product from 8082.

7. Place Order (The Test)

- **Method:** POST
- **URL:** http://localhost:8083/api/orders/place

- **Body (JSON):**

JSON

```
{
  "userName": "sai2004",
  "productId": 1,
  "quantity": 2
}
```

- **Expected Response:**

JSON

```
{
  "orderId": 1,
  "customerId": 4,
  "productId": 1,
  "unitPriceAtPurchase": 4900.0,
  "totalOrderAmount": 9800.0,
  "orderStatus": "CONFIRMED"
}
```

8. Update Order Status

- **Method:** PATCH
- **URL:** http://localhost:8083/api/orders/1/status
- **Body (JSON):**

JSON

```
{
  "newStatus": "PACKED"
}
```

NOW IMPLEMENTING SHIPMENT PROCESS FOR ORDER SERVICE AND LOGISTICS SERVICE

1. Place the Order

Status: CONFIRMED

- **Method:** POST
- **URL:** http://localhost:8083/api/orders/place
- **Body (JSON):**

JSON

```
{
   "userName": "Ahamed M",
   "productId": 101,
   "quantity": 1
}
```

- Result: Order is created in the database. orderId is generated (e.g., 1).

---

2. Move to Packing

Status: PACKED

- **Method:** PATCH
- **URL:** http://localhost:8083/api/orders/1/status
- **Body (JSON):**

JSON

```
{
   "newStatus": "PACKED"
}
```

- Result: Internal warehouse preparation. No communication with port 8086 yet.

---

3. The Integration Trigger (The Dispatch)

Status: SHIPPED

- **Method:** PATCH
- **URL:** http://localhost:8083/api/orders/1/status
- **Body (JSON):**

JSON

```
{
   "newStatus": "SHIPPED"
}
```

4. Tracking the Order (The Enrichment)

Now, you act as the customer checking the app.

- **Method:** GET
- **URL:** http://localhost:8083/api/orders/1
- **Expected Response (Port 8083 calls 8086 behind the scenes):**

JSON

```
{
   "orderId": 1,
   "orderStatus": "SHIPPED",
   "totalOrderAmount": 750.0,
   "trackingUrl": "https://mock-delhivery/track",
   "carrier": "MOCK_DELHIVERY",
   "createdAt": "2026-04-08T11:45:00"
}
```

---

5. Delivery Confirmation

Status: DELIVERED

- **Method:** PATCH
- **URL:** http://localhost:8083/api/orders/1/status
- **Body (JSON):**

JSON

```
{
   "newStatus": "DELIVERED"
}
```